Теперь прведем интересный экспримент.
На python у меня есть архитектура которую мы можем повоторит на c#:

```python
class EncoderDecoder(nn.Module):
    def __init__(self, encoder_hidden_dim, decoder_hidden_dim, vocab_size):
        super(EncoderDecoder, self).__init__()

        # 1. Энкодер принимает эмбеддинг токена (размер hidden_dim) -> выдает hidden_dim
        self.encoder = nn.LSTM(encoder_hidden_dim, encoder_hidden_dim, batch_first=True)

        # 2. ИСПРАВЛЕНО: Декодер принимает конкатенацию (embedding + encoder_output) -> размерность 1000
        self.decoder = nn.LSTM(encoder_hidden_dim + encoder_hidden_dim, decoder_hidden_dim, batch_first=True)

        # 3. Слой эмбеддингов
        self.embedding = nn.Embedding(vocab_size, encoder_hidden_dim)

        # 4. Финальная проекция в размерность словаря для расчета CrossEntropyLoss
        self.fc_out = nn.Linear(decoder_hidden_dim, vocab_size)

    def forward(self, source, target, hidden=None):
        # --- Энкодер ---
        embedded = self.embedding(source)
        encoder_outputs, next_hidden = self.encoder(embedded, hidden)

        # --- Декодер ---
        # Склеиваем эмбеддинги таргета и выходы энкодера по последней оси (dim=-1)
        # Получаем размерность: [1, 499, 1000]
        decoder_inputs = torch.cat((self.embedding(target[:, 1:]), encoder_outputs[:, :-1, :]), dim=-1)

        # Прогон через декодер
        decoder_outputs, (dec_hidden, dec_cell) = self.decoder(decoder_inputs)

        # Проекция в токены словаря: [1, 499, 7622]
        outputs = self.fc_out(decoder_outputs)

        # Возвращаем результат и состояние энкодера для передачи в следующее окно по цепочке
        return outputs, next_hidden
```

```python
VOCAB_SIZE=6565 #токенезатор уменьшился
ENCODER_HIDDEN_DIM = 500
DECODER_HIDDEN_DIM = 500
BATCH_SIZE = 8            # Теперь можно ставить больше 1 (например, 2 или 4 в зависимости от VRAM)
BLOCK_TOKENS = 1500
SAVE_EVERY_ITER = 2000    # Интервал сохранения прогресса
K_ITERATIONS = 200        # Интервал вывода лосса в консоль

DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
DATASET_PATH = r"C:\LLModels\Gemma4_E2B\fulldataset.bin" # Укажите ваш путь к новому файлу
```

```python
    dataloader = StreamDataLoader(DATASET_PATH, batch_size=BATCH_SIZE, block_tokens=BLOCK_TOKENS)

    model = EncoderDecoder(
        encoder_hidden_dim=ENCODER_HIDDEN_DIM,
        decoder_hidden_dim=DECODER_HIDDEN_DIM,
        vocab_size=VOCAB_SIZE
    )
    model.to(DEVICE)

    optimizer = optim.AdamW(model.parameters(), lr=1e-4)
    criterion = nn.CrossEntropyLoss()

    # 2. Проверка и загрузка сохраненного состояния (Чекпоинта)
    start_batch_idx = 0
    best_loss = float("inf")

    if os.path.exists(STATE_JSON_PATH) and os.path.exists(MODEL_PATH):
        try:
            with open(STATE_JSON_PATH, "r") as f:
                state = json.load(f)
            start_batch_idx = state.get("current_batch_index", 0) + 1
            best_loss = state.get("best_loss", float("inf"))

            # Загружаем веса
            model.load_state_dict(torch.load(MODEL_PATH, map_location=DEVICE, weights_only=True))
            print(f"🔄 Успешно восстановлено! Продолжаем с батча #{start_batch_idx}. Текущий лучший лосс: {best_loss:.4f}")
        except Exception as e:
            print(f"⚠️ Не удалось прочесть файл конфигурации, обучение начнется с нуля. Ошибка: {e}")
            start_batch_idx = 0

    model.train()
    running_loss = 0.0
    global_step = start_batch_idx

    # Мы не тратим время на пропуск циклом `for` — даталоадер сразу прыгнет на `start_batch_idx` через seek()
    total_batches = len(dataloader)

    # Запускаем прогресс-бар от текущего индекса до конца датасета
    pbar = tqdm(initial=start_batch_idx, total=total_batches, desc="Обучение по батчам")

    for batch_idx in range(start_batch_idx, total_batches):
        # Быстрая потоковая загрузка батча
        encoder_inputs, decoder_targets = dataloader.get_batch(batch_idx)

        # Переносим на GPU
        encoder_inputs = encoder_inputs.to(DEVICE)
        decoder_targets = decoder_targets.to(DEVICE)

        optimizer.zero_grad()

        # Полноценная последовательность: сбрасываем hiden_state в None для начала новой независимой пары блоков
        hidden = None
```

```python

        # Полноценная последовательность: сбрасываем hiden_state в None для начала новой независимой пары блоков
        hidden = None

        # Прогон через вашу encoder-decoder модель
        # inputs = encoder_inputs (1500 токенов), targets = decoder_targets (1500 токенов)
        outputs, _ = model(encoder_inputs, decoder_targets, hidden)

        # Выравниваем размерности для CrossEntropyLoss (учитываем внутренний срез [:, 1:] в декодере)
        loss = criterion(outputs.view(-1, VOCAB_SIZE), decoder_targets[:, :-1].contiguous().view(-1))
        loss.backward()

        torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
        optimizer.step()

        running_loss += loss.item()
        global_step += 1

        # Печать промежуточного лосса (каждые K итераций)
        if global_step % K_ITERATIONS == 0:
            avg_loss = running_loss / K_ITERATIONS
            pbar.write(f"[Батч {batch_idx}] Промежуточный лосс: {avg_loss:.4f}")
            running_loss = 0.0

            # Если лосс стал лучше, обновляем минимальное значение (но веса железно сохраняем каждые 2000 шагов)
            if avg_loss < best_loss:
                best_loss = avg_loss

        # Жесткое сохранение прогресса и весов каждые 2000 итераций
        if global_step % SAVE_EVERY_ITER == 0:
            pbar.write(f"💾 Достигнута итерация {global_step}. Сохранение весов и состояния...")

            # Сохраняем модель
            torch.save(model.state_dict(), MODEL_PATH)

            # Сохраняем метаданные в JSON
            state_to_save = {
                "current_batch_index": batch_idx,
                "global_step": global_step,
                "best_loss": best_loss
            }
            with open(STATE_JSON_PATH, "w") as f:
                json.dump(state_to_save, f, indent=4)

        pbar.update(1)
```

```python
class StreamDataLoader:
    def __init__(self, file_path, batch_size=4, block_tokens=1500):
        """
        Потоковый DataLoader для бинарных файлов (int32).
        Каждая запись состоит из двух блоков по block_tokens (энкодер + декодер).
        Суммарно: block_tokens * 2 чисел int32 на один элемент датасета.
        """
        self.file_path = file_path
        self.batch_size = batch_size
        self.block_tokens = block_tokens

        # 1 элемент = блок энкодера (1500) + блок декодера (1500) = 3000 чисел int32
        self.element_tokens = block_tokens * 2
        self.element_bytes = self.element_tokens * 4  # int32 занимает 4 байта

        if not os.path.exists(file_path):
            raise FileNotFoundError(f"Файл датасета не найден: {file_path}")

        # Вычисляем точное количество элементов на основе размера файла
        self.file_size = os.path.getsize(file_path)
        self.total_elements = self.file_size // self.element_bytes
        self.total_batches = self.total_elements // self.batch_size

        print(f"📊 Датасет: {file_path}")
        print(f"   Размер: {self.file_size / (1024**3):.2f} ГБ")
        print(f"   Всего элементов (строк): {self.total_elements}")
        print(f"   Всего батчей (при batch_size={batch_size}): {self.total_batches}")

    def __len__(self):
        return self.total_batches
```

```python
    def get_batch(self, batch_index):
        """
        Быстрое считывание конкретного батча из файла с помощью seek()
        """
        # Если вышли за предель датасета — зацикливаем (опционально)
        if batch_index >= self.total_batches:
            batch_index = batch_index % self.total_batches

        # Вычисляем байтовое смещение начала батча
        start_element = batch_index * self.batch_size
        byte_offset = start_element * self.element_bytes

        encoder_batch = []
        decoder_batch = []

        with open(self.file_path, "rb") as f:
            # Мгновенно прыгаем на нужную позицию в файле
            f.seek(byte_offset)

            # Читаем данные для всего батча за один раз
            bytes_to_read = self.batch_size * self.element_bytes
            chunk = f.read(bytes_to_read)

            # Превращаем байты в массив int32
            data = np.frombuffer(chunk, dtype=np.int32)

            # Разделяем массив на элементы и блоки энкодера/декодера
            for i in range(self.batch_size):
                elem_start = i * self.element_tokens

                enc_block = data[elem_start : elem_start + self.block_tokens]
                dec_block = data[elem_start + self.block_tokens : elem_start + self.element_tokens]

                encoder_batch.append(enc_block)
                decoder_batch.append(dec_block)

        # Возвращаем тензоры в формате [Batch, Sequence_Length]
        return (
            torch.tensor(np.array(encoder_batch), dtype=torch.long),
            torch.tensor(np.array(decoder_batch), dtype=torch.long)
        )
```

Нам осталось написать dataloader и собрать элементы nnclglob в правильную последовательность.
Помимо Lstm и Linear нам понядобяться слдующие блоки:

```
104
105         /// <summary>
106         /// Глобальная безопасная операция Softmax (Forward).
107         /// Рассчитана на работу со скорами внимания размерности [B * H * L, SEQ_LEN].
108         /// </summary>
            Ссылок: 2
109         public Tensor Softmax(Tensor x, int dim = -1)...
152         /// <summary>
153         /// Обратный проход глобальной операции Softmax.
154         /// Использует уже вычисленный forward-выход тензора Y для экономии памяти VRAM.
155         /// </summary>
```

```
    /// <summary>
    /// Глобальная перестановка осей тензора (Аналог x.permute(...) или x.transpose(...)).
    /// Поддерживает тензоры любой размерности.
    /// </summary>
    Ссылок: 2
    public unsafe Tensor Permute(Tensor x, int[] axes)...

    #endregion
    #region BMM
    // Добавьте эту переменную в начало класса NnClGlob к остальным ядрам:
```

```
    /// <summary>
    /// Пакетное матричное умножение (Batch Matrix Multiplication).
    /// Ожидает строго 3D-тензоры: A [Batch, M, K], B [Batch, K, N] -> Выход [Batch, M, N].
    /// </summary>
    /// <summary>
    /// Пакетное матричное умножение с поддержкой Batch Broadcasting для весов модели.
    /// </summary>
    Ссылок: 5
    public Tensor Bmm(Tensor a, Tensor b)...
```

```
    /// <summary>
    /// Выборка эмбеддингов по ID токенов (Forward).
    /// </summary>
    /// <param name="xTokensBuf">Низкоуровневый OpenCL буфер, хранящий целочисленные ID токенов (int32).</param>
    /// <param name="weight">Тензор весов эмбеддингов формы [VocabSize, EmbeddingDim].</param>
    /// <param name="totalTokens">Общее количество токенов в батче (Batch * SeqLen).</param>
    /// <returns>Выходной тензор формы [Batch * SeqLen, EmbeddingDim] или эквивалентной многомерной формы.</returns>
    Ссылок: 0
    public Tensor Embedding(NOpenCL.Buffer xTokensBuf, Tensor weight, int totalTokens)
    {
        if (weight.Shape.Length != 2)
            throw new ArgumentException("Матрица весов Embedding должна быть строго 2D тензором [VocabSize, EmbeddingDim].");
```

```
    /// <summary>
    /// Вычисляет MSE Loss и возвращает скаляр ошибки, регистрируя шаг в Autograd.
    /// </summary>
    Ссылок: 6
    public Tensor MseLoss(Tensor pred, Tensor target)...

    /// <summary>
    /// Градиент для MSE Loss: dL/dPred = 2 * (Pred - Target) / Size
```

В c# мы можем применить хитрую оптимизацию используя Marshal.AlloCoTaskMem и IntPtr поскольку все блоки имеют фиксированную длину!
Также посмкольку сохранение весов пока не реализованно, обойдемся простым логом ошибки и средней скорости обработки (элементов секунд) на каждые 100 элементов.